UNIVERSIDADE DE SÃO PAULO
ESCOLA DE ENGENHARIA DE SÃO CARLOS

Caio Wingeter de Castilho

Otimização de Baixo Nível em Vision Transformers: Desenvolvimento de Kernels de Inferência Customizados para Eficiência de SRAM

São Carlos

2026

Caio Wingeter de Castilho

Otimização de Baixo Nível em Vision Transformers: Desenvolvimento de Kernels de Inferência Customizados para Eficiência de SRAM

Monografia apresentada ao Curso de Engenharia Aeronáutica, da Escola de Engenharia de São Carlos da Universidade de São Paulo, como parte dos requisitos para obtenção do título de Engenheiro Aeronáutico.

Orientador: Prof. Dr. Glauco Caurin

VERSÃO CORRIGIDA

São Carlos

2026

É possível elaborar a ficha catalográfica em LaTeX ou incluir a fornecida pela Biblioteca. Para tanto observe a programação contida nos arquivos USPSC-modelo.tex e fichacatalografica.tex e/ou gere o arquivo fichacatalografica.pdf.

A biblioteca da sua Unidade lhe fornecerá um arquivo PDF com a ficha catalográfica definitiva, que deverá ser salvo como fichacatalografica.pdf no diretório do seu projeto.

ERRATA

A errata é um elemento opcional, que consiste de uma lista de erros da obra, precedidos pelas folhas e linhas onde eles ocorrem e seguidos pelas correções correspondentes. Deve ser inserida logo após a folha de rosto e conter a referência do trabalho para facilitar sua identificação, conforme a ABNT NBR 14724 (**??**).

Modelo de Errata:

CASTILHO, C. W. **Otimização de Baixo Nível em Vision Transformers: Desenvolvimento de Kernels de Inferência Customizados para Eficiência de SRAM**. 2026. 55 p. Monografia (Trabalho de Conclusão de Curso) - Escola de Engenharia de São Carlos, Universidade de São Paulo, São Carlos, 2026.

ERRATA

Folha	**Linha**	**Onde se lê**	**Leia-se**
1	10	auto-conclavo	autoconclavo

Folha de aprovação em conformidade
com o padrão definido
pela Unidade.
No presente modelo consta como
folhadeaprovacao.pdf

Este trabalho é dedicado à todos que me apoiaram
e me deram o suporte durante esse percurso.

AGRADECIMENTOS

Dedico este trabalho a todos que estiveram ao meu lado ao longo dessa caminhada, oferecendo apoio, força e incentivo para que eu pudesse superar cada desafio encontrado no percurso.

Agradeço, de forma especial, à minha companheira Luana, por todo o carinho, paciência e apoio incondicional ao longo dessa jornada. Aos meus pais, André e Carolina, que, apesar de todas as dificuldades, nunca deixaram de acreditar em mim e sempre se esforçaram ao máximo para que eu pudesse alcançar meus sonhos. Ao meu irmão Breno, que esteve ao meu lado em todos os momentos, não apenas como irmão, mas como meu melhor amigo.

"Líderes são mestres em encontrar o sentido e aprendizado em todo tipo de experiência de vida."

(Jucá, 2014, p. 94)

RESUMO

CASTILHO, C. W. **Otimização de Baixo Nível em Vision Transformers: Desenvolvimento de Kernels de Inferência Customizados para Eficiência de SRAM**. 2026. 55 p. Monografia (Trabalho de Conclusão de Curso) - Escola de Engenharia de São Carlos, Universidade de São Paulo, São Carlos, 2026.

Este trabalho aborda a otimização de baixo nível de kernels de inferência aplicados a modelos de visão computacional, com foco específico na arquitetura Vision Transformer (ViT). O objetivo principal consiste em mitigar o gargalo de transferência de dados entre a memória de alta largura de banda (HBM) e os processadores de fluxo (SM) da GPU, fenômeno conhecido como "Memory Wall". A metodologia proposta fundamenta-se no desenvolvimento e implementação de um kernel de atenção fundido (fused attention) utilizando a linguagem de programação de baixo nível OpenAI Triton. A abordagem técnica explora estratégias de tiling e gerenciamento manual da memória estática de acesso aleatório (SRAM), permitindo que operações de produto escalar, softmax e ponderação de valores ocorram em um único ciclo de leitura da memória global. Para a validação dos resultados, realizou-se uma análise comparativa de desempenho entre o kernel customizado e a implementação padrão (Eager mode) do framework PyTorch, utilizando ferramentas de diagnóstico de hardware como o NVIDIA Nsight Compute e o Modelo Roofline. Os resultados experimentais demonstram uma redução significativa na latência de inferência e uma diminuição no tráfego de dados no barramento de memória, deslocando a execução do kernel de um estado limitado por memória (memory-bound) para um estado limitado por processamento (compute-bound). Conclui-se que a programação de kernels de baixo nível e a fusão de operadores são requisitos essenciais para a escalabilidade de modelos de inteligência artificial de larga escala, proporcionando ganhos críticos de eficiência energética e throughput necessários para sistemas de produção de alta disponibilidade.

Palavras-chave: Visão Computacional. Vision Transformers. Otimização de Kernels. OpenAI Triton. Engenharia de Performance.

ABSTRACT

CASTILHO, C. W. **Hardware-Aware Optimization of Vision Transformers: Implementing High-Performance Inference Kernels for SRAM Efficiency**. 2026. 55 p. Monograph (Conclusion Course Paper) - Escola de Engenharia de São Carlos, Universidade de São Paulo, São Carlos, 2026.

This work addresses the low-level optimization of inference kernels applied to computer vision models, with a specific focus on the Vision Transformer (ViT) architecture. The primary objective is to mitigate the data transfer bottleneck between High Bandwidth Memory (HBM) and the GPU's Streaming Multiprocessors (SM), a phenomenon commonly known as the "Memory Wall." The proposed methodology is based on the development and implementation of a fused attention kernel using the OpenAI Triton low-level programming language. The technical approach explores tiling strategies and manual management of Static Random Access Memory (SRAM), enabling dot-product, softmax, and value weighting operations to occur within a single global memory read cycle. For performance validation, a comparative analysis was conducted between the custom kernel and the standard PyTorch Eager mode implementation, utilizing hardware diagnostic tools such as NVIDIA Nsight Compute and the Roofline Model. Experimental results demonstrate a significant reduction in inference latency and a decrease in memory bus traffic, shifting the kernel execution from a memory-bound state to a compute-bound state. The study concludes that low-level kernel programming and operator fusion are critical requirements for the scalability of large-scale AI models, providing essential gains in energy efficiency and throughput necessary for high-availability production systems.

Keywords: Computer Vision. Vision Transformers. Kernel Optimization. OpenAI Triton. Performance Engineering.

LISTA DE FIGURAS

Figura 1 – Arquitetura do Vision Transformer (ViT) 31
Figura 2 – Mecanismos de atenção do Transformer 32
Figura 3 – Hierarquia de memória com largura de banda e capacidade 33
Figura 4 – Esquema do FlashAttention e comparação de tempo de execução . . . 35
Figura 5 – Acentuação (modo texto - LaTeX) 55

LISTA DE TABELAS

LISTA DE QUADROS

LISTA DE ABREVIATURAS E SIGLAS

ABNT	Associação Brasileira de Normas Técnicas
EESC	Escola de Engenharia de São Carlos
GPU	Graphics Processing Unit
HBM	High Bandwidth Memory
HPC	High Performance Computing
ML	Machine Learning
PDF	Portable Document Format
SM	Streaming Multiprocessors
SRAM	Static Random Access Memory
TCC	Trabalho de Conclusão de Curso
USP	Universidade de São Paulo
ViT	Vision Transformer

LISTA DE SÍMBOLOS

Q	Matriz *Query*
K	Matriz *Key*
V	Matriz *Value*
K^T	Transposta da matriz K
d_k	Dimensionalidade dos vetores de chave
N	Número de *patches* (tokens) da entrada
$O(N^2)$	Complexidade computacional quadrática em função de N

SUMÁRIO

1 INTRODUÇÃO 29
1.1 Contexto e delimitação do tema 29
1.2 Problema de pesquisa 29
1.3 Objetivos 30

2 DESENVOLVIMENTO 31
2.1 Vision Transformers e a Mecânica de Atenção 31
2.1.1 Por que Autoatenção? 32
2.2 Arquitetura de GPU e a "Parede de Memória" (Memory Wall) . . . 33
2.3 OpenAI Triton 34
2.4 Fusão de Operadores e Flash Attention 34

3 METODOLOGIA 37
3.1 Ambiente Experimental 37
3.2 Design do Kernel Customizado 37
3.3 Implementação do Softmax Online 37
3.4 Protocolo de Avaliação 37

4 RESULTADOS E DISCUSSÕES 39
4.1 Análise Comparativa de Latência 39
4.2 Profiling de Baixo Nível 39
4.3 Modelo Roofline 39

5 CONCLUSÃO 41

REFERÊNCIAS 43

APÊNDICES 45

APÊNDICE A – COMPARAÇÃO ENTRE SOFTMAX PADRÃO E SOFTMAX COM TRITON 47
A.1 Implementação padrão em PyTorch 47
A.2 Implementação otimizada com Triton 48
A.3 Comparação técnica 50

ANEXOS 51

ANEXO A – EXEMPLO DE ANEXO 53

ANEXO B – ACENTUAÇÃO (MODO TEXTO - LaTeX) 55

1 INTRODUÇÃO

O avanço exponencial dos modelos de Aprendizado de Máquina (ML), particularmente na área de Visão Computacional, tem sido impulsionado pela transição de arquiteturas convolucionais para os Vision Transformers (ViTs) (Dosovitskiy *et al.*, 2021). Diferente das convoluções, que operam em domínios locais, os ViTs utilizam o mecanismo de autoatenção (Self-Attention) para capturar dependências globais em uma imagem (Vaswani *et al.*, 2023). No entanto, essa capacidade analítica superior impõe um custo computacional elevado e uma demanda intensiva de memória, apresentando desafios significativos para a implantação desses modelos em ambientes de produção de alta performance (Dao *et al.*, 2022).

1.1 Contexto e delimitação do tema

Atualmente, o gargalo da computação de alto desempenho (HPC) em Inteligência Artificial não reside apenas na capacidade aritmética das Unidades de Processamento Gráfico (GPUs), mas na eficiência da movimentação de dados. Este fenômeno, conhecido como Memory Wall, ocorre quando a velocidade de transferência entre a Memória de Alta Largura de Banda (HBM) e os Processadores de Fluxo (SM) da GPU é insuficiente para manter os núcleos de processamento (Tensor Cores) ocupados (Wulf; McKee, 1996).

No desenvolvimento de kernels de inferência, a maioria dos frameworks de alto nível, como PyTorch e TensorFlow, utiliza implementações genéricas. Embora versáteis, essas implementações frequentemente realizam múltiplos acessos à memória global para operações sequenciais. Esta pesquisa delimita-se à otimização de baixo nível desses processos, focando na camada de atenção de Vision Transformers, utilizando a linguagem de programação de kernels OpenAI Triton para maximizar o aproveitamento da hierarquia de memória do hardware (Tillet; Kung; Cox, 2019).

1.2 Problema de pesquisa

A implementação padrão da operação de atenção em ViTs possui uma complexidade quadrática em relação ao número de patches da imagem (Vaswani *et al.*, 2023; Dosovitskiy *et al.*, 2021). Durante a inferência, a necessidade de armazenar e ler matrizes intermediárias de atenção da memória global (VRAM) gera uma latência considerável e um consumo energético ineficiente (Dao *et al.*, 2022). Surge, portanto, a questão: como o desenvolvimento de kernels customizados e a aplicação de técnicas de fusão de operadores podem reduzir a dependência da memória global e aproximar a performance dos modelos de visão do limite teórico do hardware?

1.3 Objetivos

O objetivo central deste trabalho é desenvolver, implementar e validar um kernel de inferência de alto desempenho para a operação de Self-Attention em Vision Transformers.

Sendo seus objetivos específicos:

- Desenvolver um kernel fundido (fused kernel) utilizando OpenAI Triton que integre as operações de produto escalar, softmax e ponderação de valores em um único ciclo de execução na SRAM;
- Implementar estratégias de tiling para otimizar o carregamento de blocos de dados nos registradores da GPU;
- Realizar um profiling comparativo entre a implementação proposta e o baseline do PyTorch, utilizando métricas de latência, throughput (TFLOPS) e utilização de banda de memória;
- Analisar a eficiência do kernel desenvolvido através do Modelo Roofline para identificar gargalos operacionais remanescentes.

2 DESENVOLVIMENTO

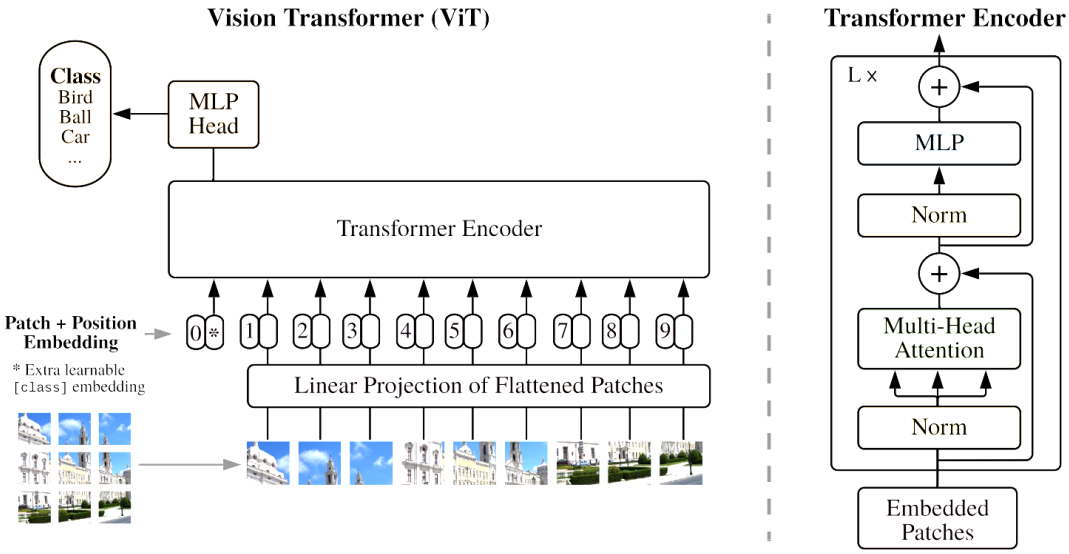
Neste capítulo, apresentam-se os conceitos fundamentais para a compreensão da otimização de kernels de inferência. A discussão abrange desde a arquitetura dos Vision Transformers até as minúcias da hierarquia de memória em Unidades de Processamento Gráfico (GPUs), culminando nas ferramentas de programação de baixo nível.

2.1 Vision Transformers e a Mecânica de Atenção

Os Vision Transformers (ViT) representam uma mudança de paradigma na visão computacional ao transpor a arquitetura de autoatenção, originalmente concebida para processamento de linguagem natural (Vaswani *et al.*, 2023), para o domínio de imagens. Segundo Dosovitskiy *et al.* (2021), o ViT decompõe uma imagem em patches fixos, os quais são tratados como sequências de tokens, permitindo que o modelo aprenda relações globais entre diferentes regiões da cena desde as camadas iniciais.

Como referência visual da arquitetura, a Figura 1 apresenta o fluxo do ViT, desde a tokenização em patches até o codificador Transformer e a cabeça de classificação.

Figura 1 – Arquitetura do Vision Transformer (ViT)



Fonte: Adaptado de Dosovitskiy *et al.* (2021).

O cerne dessa arquitetura é o mecanismo de autoatenção, cujos componentes fundamentais, o Scaled Dot-Product Attention e sua extensão para o Multi-Head Attention,

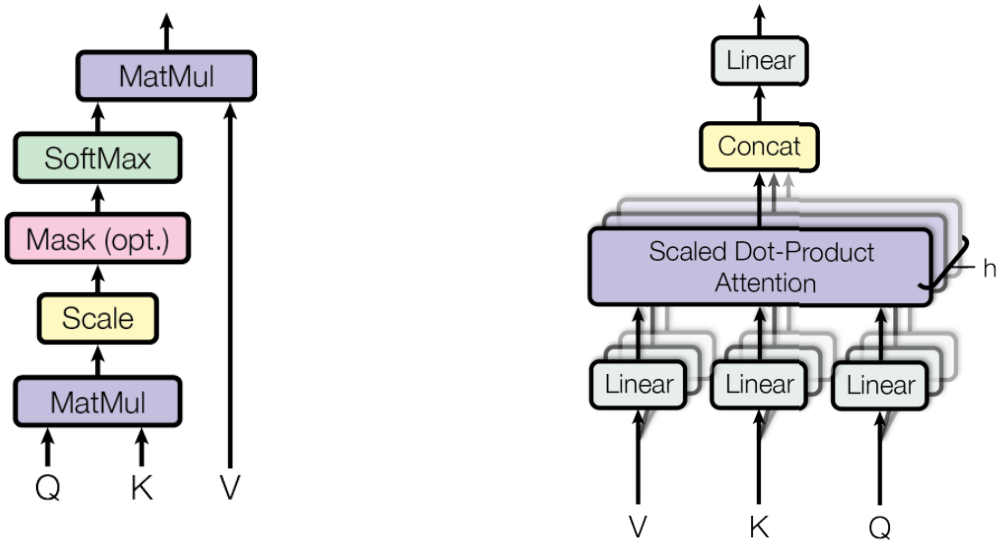
são ilustrados na Figura 2. Matematicamente, a operação de atenção é definida pela Equação 2.1:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{2.1}$$

Figura 2 – Mecanismos de atenção do Transformer

(a) Scaled Dot-Product Attention

(b) Multi-Head Attention



Fonte: Vaswani *et al.* (2023).

Onde Q, K e V representam as matrizes de Query, Key e Value. Embora eficaz, essa operação apresenta uma complexidade computacional $O(N^2)$, onde N é o número de patches. Como apontam Dao *et al.* (2022), o gargalo dessa operação em GPUs modernas não é o poder computacional bruto, mas sim a latência de memória gerada pela leitura e escrita de matrizes intermediárias de atenção.

2.1.1 Por que Autoatenção?

No trabalho original dos Transformers, Vaswani *et al.* (2023) justificam o uso da autoatenção comparando-a com camadas recorrentes e convolucionais para o mapeamento de uma sequência de entrada em outra de mesmo comprimento. A análise é guiada por três critérios centrais: (i) complexidade computacional por camada, (ii) grau de paralelização e (iii) comprimento do caminho entre dependências de longo alcance.

Sob essa ótica, a autoatenção apresenta uma vantagem estrutural importante: todas as posições da sequência interagem diretamente em uma única camada, com número constante de operações sequenciais. Em contraste, arquiteturas recorrentes exigem dependência

temporal passo a passo, com custo sequencial proporcional ao comprimento da sequência. Na prática, isso reduz o paralelismo possível em GPU e limita o throughput.

Também no aspecto de dependências de longo alcance, a autoatenção encurta o caminho de propagação de informação entre dois tokens para ordem constante. Em arquiteturas recorrentes, esse caminho cresce linearmente com o tamanho da sequência; em convolucionais, depende da largura do kernel e da profundidade da pilha de camadas. Essa diferença é relevante porque caminhos menores tendem a facilitar o aprendizado de relações globais.

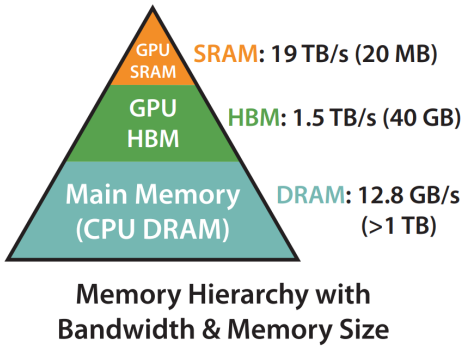
Em termos de custo assintótico, a autoatenção densa envolve tipicamente $O(n^2d)$, enquanto camadas recorrentes escalam como $O(nd^2)$ e convoluções dependem de k (largura do kernel). Conforme discutido por Vaswani *et al.* (2023), quando $n < d$, cenário comum em representações de linguagem e compatível com várias configurações de tokens em visão, a autoatenção tende a ser computacionalmente mais favorável que abordagens recorrentes, além de ser mais paralelizável.

Por fim, os autores destacam que versões restritas de autoatenção (janela local de tamanho r) podem reduzir custo para sequências muito longas, ao custo de aumentar o caminho máximo para $O(n/r)$. Esse compromisso custo–alcance é particularmente útil para conectar a formulação teórica da atenção com o problema de engenharia tratado neste trabalho: minimizar movimentação de dados e latência de memória sem perder capacidade de modelar dependências globais.

2.2 Arquitetura de GPU e a "Parede de Memória" (Memory Wall)

Para entender a necessidade de otimização de baixo nível, é preciso distinguir os diferentes níveis de memória em uma GPU. A memória principal, conhecida como High Bandwidth Memory (HBM), possui alta capacidade, porém latência significativamente superior à memória estática interna do chip, a SRAM.

Figura 3 – Hierarquia de memória com largura de banda e capacidade



Fonte: Adaptado de Dao *et al.* (2022).

O desequilíbrio entre a velocidade de processamento e a velocidade de transferência de dados é o que a literatura denomina como Memory Wall. Sobre este desafio, Wulf e McKee (1996) observam:

> [...] a disparidade entre a velocidade dos processadores e a velocidade da memória principal tem crescido exponencialmente. Se essa tendência continuar, a performance dos sistemas será limitada não pela frequência de clock do processador, mas pela taxa na qual os dados podem ser entregues a ele (Wulf; McKee, 1996) (tradução do autor).

Dessa forma, otimizar um kernel significa maximizar o tempo que o dado permanece na SRAM (ou Shared Memory), minimizando acessos à HBM. Em implementações padrão de bibliotecas de alto nível, cada etapa da Equação 2.1 resulta em uma escrita e leitura na HBM, o que degrada o throughput do sistema.

2.3 OpenAI Triton

Historicamente, a escrita de kernels otimizados exigia o uso de CUDA C++, uma linguagem de alta complexidade que demanda gerenciamento manual de threads e sincronização de blocos. O OpenAI Triton surge como uma alternativa de abstração que permite a escrita de kernels altamente performáticos utilizando uma sintaxe baseada em Python.

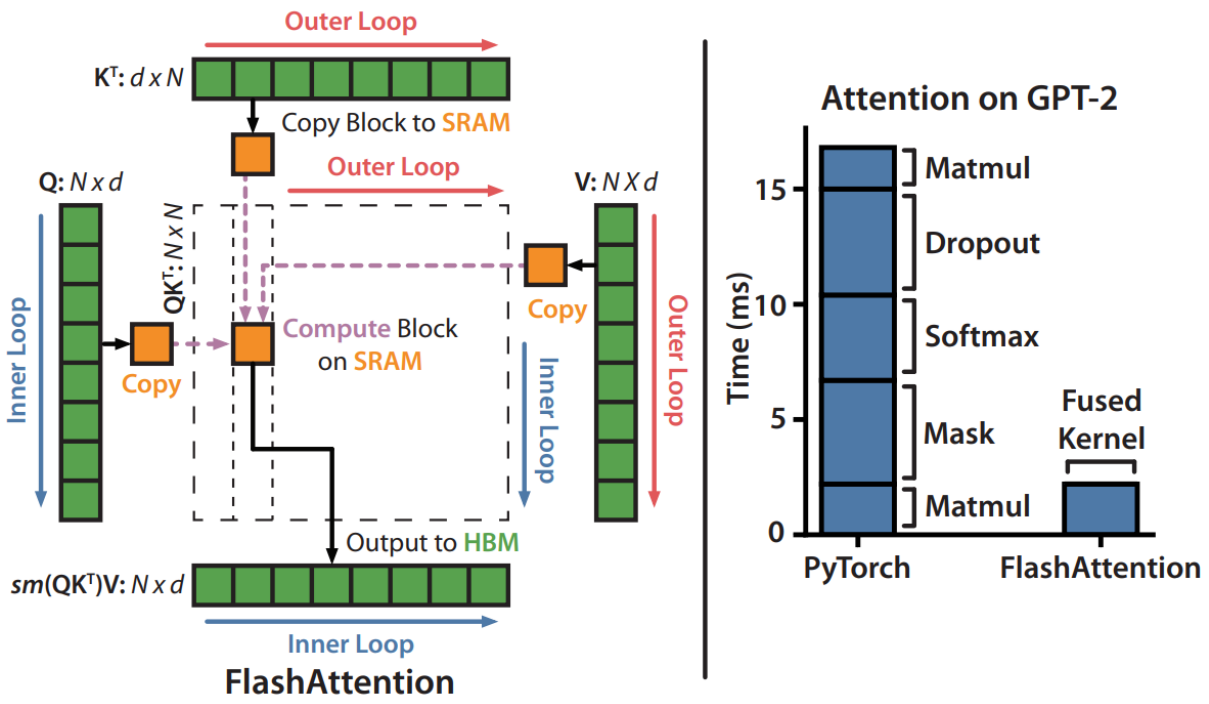
De acordo com Tillet, Kung e Cox (2019), o Triton introduz um paradigma baseado em blocos (tiles), onde o compilador é responsável por otimizar automaticamente o agendamento de tarefas e o uso dos registradores, permitindo que o desenvolvedor foque na lógica de movimentação de dados na hierarquia de memória.

2.4 Fusão de Operadores e Flash Attention

A técnica de fusão de operadores (Operator Fusion) consiste em agrupar múltiplas operações matemáticas em um único kernel executável. No contexto de modelos de visão, isso significa realizar o cálculo do Softmax e a multiplicação por V enquanto os dados de Q ainda residem nos registradores da GPU.

Como referência visual do método, a Figura 4 ilustra a hierarquia de memória, o particionamento em blocos e o ganho de desempenho obtido pelo kernel fundido do FlashAttention.

Figura 4 – Esquema do FlashAttention e comparação de tempo de execução



Fonte: Adaptado de Dao *et al.* (2022).

Esta técnica é levada ao estado da arte pelo algoritmo Flash Attention. Como sintetizado por Dao *et al.* (2022), a fusão de operadores permite que o modelo evite a materialização da matriz de atenção $N \times N$ na memória principal, resultando em acelerações significativas em relação às implementações tradicionais, reduzindo simultaneamente o consumo de memória.

3 METODOLOGIA

3.1 Ambiente Experimental

3.2 Design do Kernel Customizado

3.3 Implementação do Softmax Online

A comparação detalhada entre uma implementação padrão de *softmax* em PyTorch e uma versão otimizada com OpenAI Triton é apresentada no Apêndice A. O apêndice inclui os trechos de código e uma análise objetiva dos impactos em movimentação de memória, paralelismo e ocupação do hardware.

3.4 Protocolo de Avaliação

4 RESULTADOS E DISCUSSÕES

4.1 Análise Comparativa de Latência

4.2 Profiling de Baixo Nível

4.3 Modelo Roofline

5 CONCLUSÃO

Apresentar as conclusões correspondentes aos objetivos ou hipóteses propostos para o desenvolvimento do trabalho, podendo incluir sugestões para novas pesquisas.

REFERÊNCIAS

DAO, T. *et al.* **FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness**. 2022. Disponível em: https://arxiv.org/abs/2205.14135.

DOSOVITSKIY, A. *et al.* **An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale**. 2021. Disponível em: https://arxiv.org/abs/2010.11929.

TILLET, P.; KUNG, H. T.; COX, D. Triton: an intermediate language and compiler for tiled neural network computations. *In*: **Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages**. New York, NY, USA: Association for Computing Machinery, 2019. (MAPL 2019), p. 10–19. ISBN 9781450367196. Disponível em: https://doi.org/10.1145/3315508.3329973.

VASWANI, A. *et al.* **Attention Is All You Need**. 2023. Disponível em: https://arxiv.org/abs/1706.03762.

WULF, W.; MCKEE, S. Hitting the memory wall: Implications of the obvious. **Computer Architecture News**, v. 23, 01 1996.

APÊNDICES

APÊNDICE A – COMPARAÇÃO ENTRE SOFTMAX PADRÃO E SOFTMAX COM TRITON

Este apêndice apresenta uma comparação entre duas abordagens para cálculo de *softmax* em linhas de uma matriz: uma implementação padrão em PyTorch e uma implementação otimizada com OpenAI Triton.

A.1 Implementação padrão em PyTorch

```
import torch

import triton
import triton.language as tl
from triton.runtime import driver

DEVICE = triton.runtime.driver.active.get_active_torch_device()


def is_hip():
    return (
        triton.runtime.driver.active.get_current_target().backend == "hip"
    )


def is_cdna():
    return (
        is_hip()
        and triton.runtime.driver.active.get_current_target().arch in (
            'gfx940', 'gfx941', 'gfx942', 'gfx90a', 'gfx908'
        )
    )


def naive_softmax(x):
    """Compute row-wise softmax of X using native pytorch

    We subtract the maximum element in order to avoid overflows. Softmax is
    invariant to this shift.
```

```
    """
    # read  MN elements ; write M  elements
    x_max = x.max(dim=1)[0]
    # read MN + M elements ; write MN elements
    z = x - x_max[:, None]
    # read  MN elements ; write MN elements
    numerator = torch.exp(z)
    # read  MN elements ; write M  elements
    denominator = numerator.sum(dim=1)
    # read MN + M elements ; write MN elements
    ret = numerator / denominator[:, None]
    # in total: read 5MN + 2M elements ; wrote 3MN + 2M elements
    return ret
```

A.2 Implementação otimizada com Triton

```
properties = driver.active.utils.get_device_properties(DEVICE.index)
NUM_SM = properties["multiprocessor_count"]
NUM_REGS = properties["max_num_regs"]
SIZE_SMEM = properties["max_shared_mem"]
WARP_SIZE = properties["warpSize"]
target = triton.runtime.driver.active.get_current_target()
kernels = {}


def softmax(x):
    n_rows, n_cols = x.shape

    # The block size of each loop iteration is the smallest power of two
    # greater than the number of columns in ‘x‘
    BLOCK_SIZE = triton.next_power_of_2(n_cols)

    # Another trick we can use is to ask the compiler to use more threads
    # per row by increasing the number of warps (‘num_warps‘)
    # over which each row is distributed.
    num_warps = 8

    # Number of software pipelining stages.
    num_stages = 4 if SIZE_SMEM > 200000 else 2
```

```
# Allocate output
y = torch.empty_like(x)

# pre-compile kernel to get register usage
# and compute thread occupancy.
kernel = softmax_kernel.warmup(
    y, x, x.stride(0), y.stride(0), n_rows, n_cols,
    BLOCK_SIZE=BLOCK_SIZE, num_stages=num_stages,
    num_warps=num_warps, grid=(1, )
)
kernel._init_handles()
n_regs = kernel.n_regs
size_smem = kernel.metadata.shared

if is_hip():
    # NUM_REGS represents the number of regular purpose registers.
    # On CDNA architectures this is half of all registers available.
    NUM_GPRS = NUM_REGS
    if is_cdna():
        NUM_GPRS = NUM_REGS * 2

    # MAX_NUM_THREADS represents maximum resident threads per SM.
    MAX_NUM_THREADS = properties["max_threads_per_sm"]
    max_num_waves = MAX_NUM_THREADS // WARP_SIZE
    occupancy = min(NUM_GPRS // WARP_SIZE // n_regs, max_num_waves)
    occupancy = occupancy // num_warps
else:
    occupancy = NUM_REGS // (n_regs * WARP_SIZE * num_warps)

occupancy = min(occupancy, SIZE_SMEM // size_smem)
num_programs = NUM_SM * occupancy
num_programs = min(num_programs, n_rows)

# Create a number of persistent programs.
kernel[(num_programs, 1, 1)](
    y, x, x.stride(0), y.stride(0), n_rows, n_cols,
    BLOCK_SIZE, num_stages
)
```

```
return y
```

A.3 Comparação técnica

- **Movimentação de memória:** a versão padrão materializa tensores intermediários para cada etapa; a versão Triton busca reduzir leituras e escritas na HBM por meio de execução em blocos e reutilização de dados em SRAM.
- **Paralelismo:** na implementação Triton, parâmetros como `BLOCK_SIZE`, `num_warps` e `num_stages` são ajustados para aumentar ocupação e throughput.
- **Consciência de hardware:** o código Triton utiliza propriedades do dispositivo (SMs, registradores e memória compartilhada) para definir o número de programas persistentes, aproximando a implementação do limite arquitetural da GPU.
- **Objetivo prático:** enquanto a versão padrão prioriza simplicidade e legibilidade, a versão Triton prioriza eficiência computacional e redução da latência em inferência.

ANEXOS

ANEXO A – EXEMPLO DE ANEXO

Elemento opcional, que consiste em um texto ou documento não elaborado pelo autor, que serve de fundamentação, comprovação e ilustração, conforme a ABNT NBR 14724. (**??**).

O **ANEXO B** exemplifica como incluir um anexo em pdf.

ANEXO B – ACENTUAÇÃO (MODO TEXTO - LaTeX)

Figura 5 – Acentuação (modo texto - LaTeX)

\'a - á
\'a - à
\~a - ã
\^a - â
\'e - é
\^e - ê
\'{\i} - í
\'I - Í
\'o - ó
\~o - õ
\^o - ô
\'u - ú
\"u - ü
\c{c} - ç
\c{C} - Ç

Fonte: Comandos [...] (**??**)